IAI600 – Introduction to Applied Artificial Intelligence

Lab I: Data Exploration, Cleaning and Feature Preparation

Main topic: Chapter 2 (Literature Book) – End-to-end Machine Learning project
Dataset: California Housing Dataset
Software: Python, Jupyter Notebook, Pandas, NumPy, Matplotlib, Scikit-learn
Disclaimer: This lab material has been prepared with the assistance of Generative AI and is based on Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow; please report any errors, inaccuracies, or limitations you identify.

Objective of the laboratory: The objective of this laboratory is to develop a practical understanding of the first stages of an end-to-end machine learning project. Students will explore a real-world dataset, identify data quality issues, investigate relationships between variables, create appropriate features, and prepare the data for machine learning.
By the end of the laboratory, students should be able to explain not only *what* each operation does, but also *why* it is required before training a machine learning model.

Background

In this laboratory, we begin an end-to-end machine learning project using the California Housing dataset. The dataset contains information about housing districts in California, including variables such as median income, number of rooms, population, latitude, longitude, and median house value.

This laboratory focuses on the stages that must be completed **before training a machine learning model**:

1. Loading and understanding the dataset
2. Exploring the data
3. Creating a suitable training and test split
4. Investigating the relationships between attributes
5. Identifying and handling missing values
6. Handling categorical attributes
7. Preparing numerical attributes
8. Feature scaling
9. Building a preprocessing pipeline

Model training and model selection will be performed in the next laboratory (Lab II).

Part I – Setting up the Machine Learning Environment

Before starting the analysis, make sure that Python and the required libraries are available on your computer.

You may use either:

- Anaconda and Jupyter Notebook, or
- another Python environment for AI/ML.

Task 1.1 – Verify the Environment

Create a new Jupyter Notebook and verify that Python and the required libraries can be imported successfully.

The laboratory uses the following Python libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

Part II – Obtain and Load the Dataset

The dataset used in this laboratory is the California Housing dataset. Download or obtain the dataset provided at the Gituhb directory Here.

Load the dataset using Pandas.

```
import pandas as pd

housing = pd.read_csv("housing.csv")
```

Inspect the first few rows:

```
housing.head()
```

Then inspect the structure of the dataset:

```
housing.info()
```

Calculate descriptive statistics:

```
housing.describe()
```

Task 2.1 – Understand the Dataset

Answer the following questions:

1. How many observations are contained in the dataset?
2. How many attributes are available?
3. Which attributes are numerical?
4. Which attribute is categorical?
5. What is the target variable for the prediction problem?
6. Are there any missing values?

Part III – Exploring the Data

Before applying machine learning, it is important to understand the data.

Task 3.1 – Attribute Distributions

Create histograms for the numerical attributes. For example:

```
housing.hist(bins=50, figsize=(12, 8))
plt.show()
```

Inspect the resulting distributions.
Consider:

- Which attributes have approximately symmetric distributions?
- Which attributes are strongly skewed?
- Are there attributes with very different numerical scales?
- Are there unusually large or small values?

Part IV – Creating a Test Set

Before performing extensive data analysis and preprocessing, we should separate the data used for developing the model from data reserved for final evaluation.

Create a training and test set. For example:

```
train_set, test_set = train_test_split(
    housing,
    test_size=0.2,
    random_state=42
)
```

Note: The test set should be kept separate during model development.

Task 4.1 – Why Separate the Data?

Explain in your own words:

1. Why do we need a test set?
2. What could happen if we repeatedly use the test set while developing the model?
3. Why should preprocessing operations also be learned from the training data rather than the complete dataset?

Part V – Stratified Sampling

The median income attribute is particularly important in this dataset.

Create income categories from the median income attribute:

```
housing["income_cat"] = pd.cut(
    housing["median_income"],
    bins=[0., 1.5, 3., 4.5, 6., np.inf],
    labels=[1, 2, 3, 4, 5]
)
```

Examine the distribution:

```
housing["income_cat"].value_counts()
```

Use stratified sampling to create a test set that preserves the distribution of the income categories.

```
from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(
    n_splits=1,
    test_size=0.2,
    random_state=42
)

for train_index, test_index in split.split(
       housing, housing["income_cat"]):

    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]
```

Task 5.1 – Compare Sampling Strategies

Compare the income distribution obtained using:

- random sampling, and
- stratified sampling.

Why might stratified sampling be preferable when an attribute has an important relationship with the target variable?
NOte: After the comparison, remove the temporary `income_cat` attribute.

Part VI – Geographical Visualization

The dataset contains latitude and longitude information for each district. Create a geographical scatter plot:

```
housing.plot(
    kind="scatter",
    x="longitude",
    y="latitude",
    alpha=0.1
)
plt.show()
```

Improve the visualization by using population to represent the size of the points and median house value to represent the colour/intensity of the points.

```
housing.plot(
    kind="scatter",
    x="longitude",
    y="latitude",
    alpha=0.4,
    s=housing["population"] / 100,
    c="median_house_value",
    cmap=plt.get_cmap("jet"),
    colorbar=True
)
plt.show()
```

Task 6.1 – Interpret the Visualization

Do not focus only on producing the plot. Explain:

1. What geographical patterns can you observe?
2. Where are the high-density areas?
3. Do house values appear to vary geographically?
4. Why could geographical information be useful for predicting house prices?

Part VII – Correlation Analysis

Calculate the correlation matrix for the numerical attributes.

```
corr_matrix = housing.corr(numeric_only=True)

corr_matrix["median_house_value"].sort_values(
    ascending=False
)
```

Investigate the relationship between selected attributes and the target. For example:

```
housing.plot(
    kind="scatter",
    x="median_income",
    y="median_house_value",
    alpha=0.1
)
plt.show()
```

Task 7.1 – Interpret Correlations

Answer:

1. Which attribute has the strongest correlation with median house value?
2. Does a strong correlation necessarily mean that one attribute causes the target to change?
3. Are there attributes with weak correlation that might still be useful?
4. Can correlation alone be used to select the final machine learning model?

Part VIII – Feature Engineering

Raw attributes are not always the most useful representation of the problem. Create additional attributes that combine existing information. For example:

```
housing["rooms_per_household"] = (
    housing["total_rooms"] /
    housing["households"]
)

housing["bedrooms_per_room"] = (
    housing["total_bedrooms"] /
    housing["total_rooms"]
)
```

```
housing["population_per_household"] = (
    housing["population"] /
    housing["households"]
)
```

Recalculate the correlations and investigate whether the engineered attributes provide useful information.

Task 8.1 – Feature Engineering Discussion

Explain:

1. Why can combining existing attributes be useful?
2. Which engineered attribute appears most promising?
3. What could happen if we create too many unnecessary features?

Part IX – Handling Missing Values

Machine learning algorithms generally require numerical input without missing values. First identify missing values:

```
housing.isnull().sum()
```

Consider different strategies for dealing with missing values:

1. Remove the corresponding rows.
2. Remove the corresponding attribute.
3. Replace missing values using a statistic such as the median.

Use Scikit-learn's `SimpleImputer` to replace missing numerical values with the median.

```
imputer = SimpleImputer(strategy="median")
```

Fit the imputer using the training data only.

```
housing_num = housing.drop("ocean_proximity", axis=1)

imputer.fit(housing_num)
```

Transform the data:

```
X = imputer.transform(housing_num)
```

Task 9.1 – Compare Missing-Value Strategies

Briefly discuss the advantages and disadvantages of:

- deleting rows,
- deleting an attribute,
- replacing missing values with the median.

Question: Why is it important to calculate the median from the training data rather than from the complete dataset?

Part X – Handling Categorical Attributes

The `ocean_proximity` attribute is categorical.

Inspect its values:

```
housing["ocean_proximity"].value_counts()
```

Machine learning algorithms generally require numerical representations of categorical variables.

Use one-hot encoding:

```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()

housing_cat = housing[["ocean_proximity"]]

housing_cat_1hot = cat_encoder.fit_transform(
    housing_cat
)
```

Task 10.1 – Understand One-Hot Encoding

Explain:

1. Why cannot we simply assign the categories the values 1, 2, 3, and so on?
2. What does one-hot encoding achieve?
3. How many encoded features are created for the `ocean_proximity` attribute?

Part XI – Feature Scaling

The numerical attributes in the dataset have very different scales. For example, population and median income do not have comparable numerical ranges. Many machine learning algorithms perform better when numerical attributes are on comparable scales.
Two common approaches are:

- min-max scaling (normalization),
- standardization.

Use Scikit-learn's `StandardScaler`:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

housing_num_scaled = scaler.fit_transform(
    housing_num
)
```

Task 11.1 – Understand Feature Scaling

Answer:

1. Why can different feature scales be a problem for some ML algorithms?
2. What does standardization do?
3. Does feature scaling change the information contained in the data?
4. Is feature scaling required for every machine learning algorithm?

Part XII – Building a Preprocessing Pipeline

Performing preprocessing manually can lead to repetitive code and mistakes. Scikit-learn provides the `Pipeline` and `ColumnTransformer` classes to organize preprocessing operations.
First separate numerical and categorical attributes:

```
housing = strat_train_set.drop(
    "median_house_value",
    axis=1
)

housing_labels = strat_train_set[
    "median_house_value"
].copy()

housing_num = housing.drop(
    "ocean_proximity",
    axis=1
)

housing_cat = housing[
    ["ocean_proximity"]
]
```

Create a numerical preprocessing pipeline:

```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

Create the complete preprocessing pipeline:

```
num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs)
])
```

Apply the pipeline to the training data:

```
housing_prepared = full_pipeline.fit_transform(
    housing
)
```

Task 12.1 – Understand the Pipeline

Explain the sequence of operations performed by the pipeline.
In particular, identify:

1. Where missing numerical values are handled.
2. Where numerical features are scaled.
3. Where categorical features are encoded.
4. Why using a pipeline is preferable to manually repeating the preprocessing operations.